



UNIVERSIDAD DE GRANADA

Proyecto Final

Periféricos y Dispositivos de Interfaz Humana

Escaneo de Tickets mediante OCR con Tesseract

Pablo Soriano Fleitas

Objetivo:

La finalidad de este proyecto es intentar optimizar el reconocimiento del texto y de los caracteres que muestra un ticket mediante el empleo de la Api de OCR de Tesseract.

Imagen entrada



Primera Prueba

Código simple en el que simplemente le paso como argumento a la función de Tesseract una imagen de un recibo.

```
import cv2
import pytesseract

img = cv2.imread('./ProyectoFinalPDIH/prueba.jpeg')
text = pytesseract.image_to_string(img)

print(text)
```

Texto salida

MERGADONA, S.A.

h- 46103834

C.2 DE RONDA, 178

18003 GRANADA

TELEFONO: 958201084

06/04/2026 17: re OP : 2665399

FACTURA SIMPLIFICADA;: 2751-019- 103926

IA

oo Descripción _ Ps Unit Imp.(€)

1NEVAL 1,5. 8 — 0,27

| PLATANO

0,584 kg 2,30 €/kg 1,34

TOTAL iH 1,6

TARJETA BANCART 1,61

IVA BASE IMPONIBLE (€) CUOTA (€)

4h Le] 0,05

10% 0,25 0,02

TOTAL 1,54 0,07

TARJ. BANCARTA: RR ek ik 0592

N.C; 072253776 AUT; N45 145

AID: A0000000041010 ARC: 00

|”)

MASTERCARD

Tnporte: 1,61 € DEBIT MASTERCARD

Observamos que en la salida ya tenemos algunos resultados que se pueden comprender medianamente, aunque no termina de ser del todo acertado con fallos en varias palabras y símbolos.

Segunda Prueba

Para la segunda prueba aplico un preprocesado básico a la imagen con escala de grises junto a un blur ligero y threshold para facilitar la captación de los caracteres.

```
img = cv2.imread('./ProyectoFinalPDIH/prueba.jpeg')

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5,5), 0)
thresh = cv2.adaptiveThreshold(
    blur, 255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    11, 2
)

text = pytesseract.image_to_string(thresh)

print(text)
```

Texto salida

MERCADONA, S.A.

j~46103834--.

© DE RONDA, 178

48003 GRANADA

TELEFONO; 958201084

06/04/2026 17:45 OP :2665399

FACTURA SINPLIFICADA: 2751-019-103926

ATTN _ a

==> pestripe! Loo IP, Unit

1 NEVAL 1, aL ae Eb zt
a1 PLATANO ~ _ ;
0,584 ko: | “2a ea 18d
INTL: 7 “161
“TREN BANCAR |, 6
IVA: BASE TMONIBLE (e) ““cudta (€)
4% 129.0 49,05
10% en | 9: 0,02
TOTAL anne a "4 YG 07
TARY. BANCARTA: — Tp hik asta at 0592, \
N.C: 072253776. AUT: Nasi
AID: 40000006041010 ARGS S00
v)) :
MASTERCARD ae

Importe: 1,61 'e DEBIT MASTERCARD.

En este caso podemos observar que el preprocesado aplicado antes de llamar a la función de la Api de Tesseract ha empeorado el resultado de la salida.

Aunque el preprocesado de imagen suele emplearse para mejorar la calidad de la entrada en sistemas OCR, he observado que ciertos métodos como la binarización o el suavizado pueden degradar el rendimiento.

Esto se debe a que Tesseract incorpora su propio pipeline de preprocesado optimizado, incluyendo binarización y segmentación. Al aplicar transformaciones externas agresivas, se puede perder información relevante o introducir artefactos que dificultan el reconocimiento.

Por tanto, la mejora del OCR depende en gran medida del tipo de imagen, no existiendo un preprocesado universalmente óptimo.

Tercera Prueba

Para esta última prueba lo último que decidí probar es a quitar directamente el blur para evitar perder esos posibles pequeños detalles dependiendo de las imágenes, separando a su vez dos posibles salidas. Por un lado muestro el resultado de aplicar OCR a la imagen con el escalado de grises directamente, y por otro lado pruebo a realizarle el OCR a la imagen tras aplicarle un threshold simple.

```
img = cv2.imread('./prueba.jpeg')

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, thresh = cv2.threshold(
    gray, 150, 255,
    cv2.THRESH_BINARY
)

text1 = pytesseract.image_to_string(gray)
text2 = pytesseract.image_to_string(thresh)

print("Texto extraído de la imagen original con escala de grises:\n")
print(text1)
print("\n")
print("Texto extraído de la imagen umbralizada:\n")
print(text2)
```

Texto salida

Texto extraído de la imagen original con escala de grises:

MERGADONA, S.A.

h- 46103834

C.2 DE RONDA, 178

18003 GRANADA

TELEFONO: 958201084

06/04/2026 17: ~ OP : 2665399

FACTURA SIMPLIFICADA; 2751-019-103926

IMI

ePestrición Pa Unit Imp. (€)

1NEVAL 1,5. 83 0,27

1 PLATANO

0,584 kg 2,30 e/kg 1,34

TOTAL IH 1,61

TARJETA BANCART 1,61

IVA BASE IMPONIBLE (€) CUOTA (€)

4% 1,29 0,05

10% 0,25 0,02

TOTAL 1,54 0,07

TARJ. BANCARIA; eR Adaek kak 0532

N.C: 072253776 AUT: N45 ar

AID: AGOG0000041010 ARC: 00

)

MASTERCARD)

Importe: 1,61 € DEBIT MASTERCARD

Texto extraído de la imagen umbralizada:

MERCADONA, 5.4.

A 46103834

C9 DE RONDA, 176

THOG3 GRANADA

HEE PFE OND: JHBZ0 1084

Ob/4/7026 V4 OPS 2665399

PaclukA SIMPL IETCADA: 2/51-619- tosuz6

Aa

|

Descripcion Ps Unit Imp.fe;

1 NEVAL 1,51 0,27

1 PLATANO

0,584 kg 2,30 €/kg 1,34

TOTAL yi 1,6!

TARJETA BANCAR] 1,61

IVA BASE IMPONIBLE (€) CUOTA (€)

AY 1,29 0,05

10% 0,25 0,02

TOTAL 1,54 0,07

JARS. BANCARIA: oe dock hbk 532

N.C: 072253776 AUT: N451745

AID: AVOG0000041010 ARC: OQ

)

MASTERCARD)

Importe: 1,61 € DEBLT MASTERCARD

Podemos observar que siguen habiendo bastante fallos a la hora de detectar algunos caracteres, aunque en este caso para detectar los productos y las cantidades se puede observar una mejoría mayor que en el caso anterior con el preprocesado completo de la imagen con la binarización.

Prueba Final

Finalmente, una vez y ya he comprobado que Tesseract funciona correctamente casi incluso sin necesidad de que realice ningún preprocesado a la imagen que le quiero pasar como argumento, voy a proceder a capturar la imagen del recibo que le quiero pasar como argumento a la Api a través de la webcam de mi portátil, para así o tener que escanear el recibo desde el móvil y pasárselo al programa como pdf, sino que se haga automático tras mostrarlo a la webcam.

La idea es que se active la webcam y que cuando vea conveniente pueda pulsar para realizar una captura, y luego sobre esa captura recortar la imagen para quedarme solo con la parte del recibo y esa parte sería la que le paso a la Api de Tesseract para que la escanee.

```
drawing = False
ix, iy = -1, -1
fx, fy = -1, -1

def draw_rectangle(event, x, y, flags, param):
    global ix, iy, fx, fy, drawing

    if event == cv2.EVENT_LBUTTONDOWN:
        drawing = True
        ix, iy = x, y

    elif event == cv2.EVENT_MOUSEMOVE:
        if drawing:
            fx, fy = x, y

    elif event == cv2.EVENT_LBUTTONUP:
        drawing = False
        fx, fy = x, y

cap = cv2.VideoCapture(0)
captured = None
```

```
while True:
    ret, frame = cap.read()

    if not ret:
        print("Error al capturar")
        break

    cv2.imshow("Camara", frame)

    key = cv2.waitKey(1)

    # Captura
    if key == ord('c'):
        captured = frame.copy()
        cloned = captured.copy()

        cv2.namedWindow("Seleccion")
        cv2.setMouseCallback("Seleccion", draw_rectangle)

        while True:
            temp = cloned.copy()

            if ix != -1 and iy != -1 and fx != -1 and fy != -1:
                cv2.rectangle(temp, (ix, iy), (fx, fy), (0,255,0), 2)

            cv2.imshow("Seleccion", temp)
            k = cv2.waitKey(1)

            # Enter → confirmar selección
            if k == 13:
                break

        crop = captured[min(iy, fy):max(iy, fy), min(ix, fx):max(ix, fx)]

        cv2.imshow("Recorte", crop)

        # OCR
        crop = cv2.resize(crop, None, fx=2, fy=2, interpolation=cv2.INTER_CUBIC)
        gray = cv2.cvtColor(crop, cv2.COLOR_BGR2GRAY)

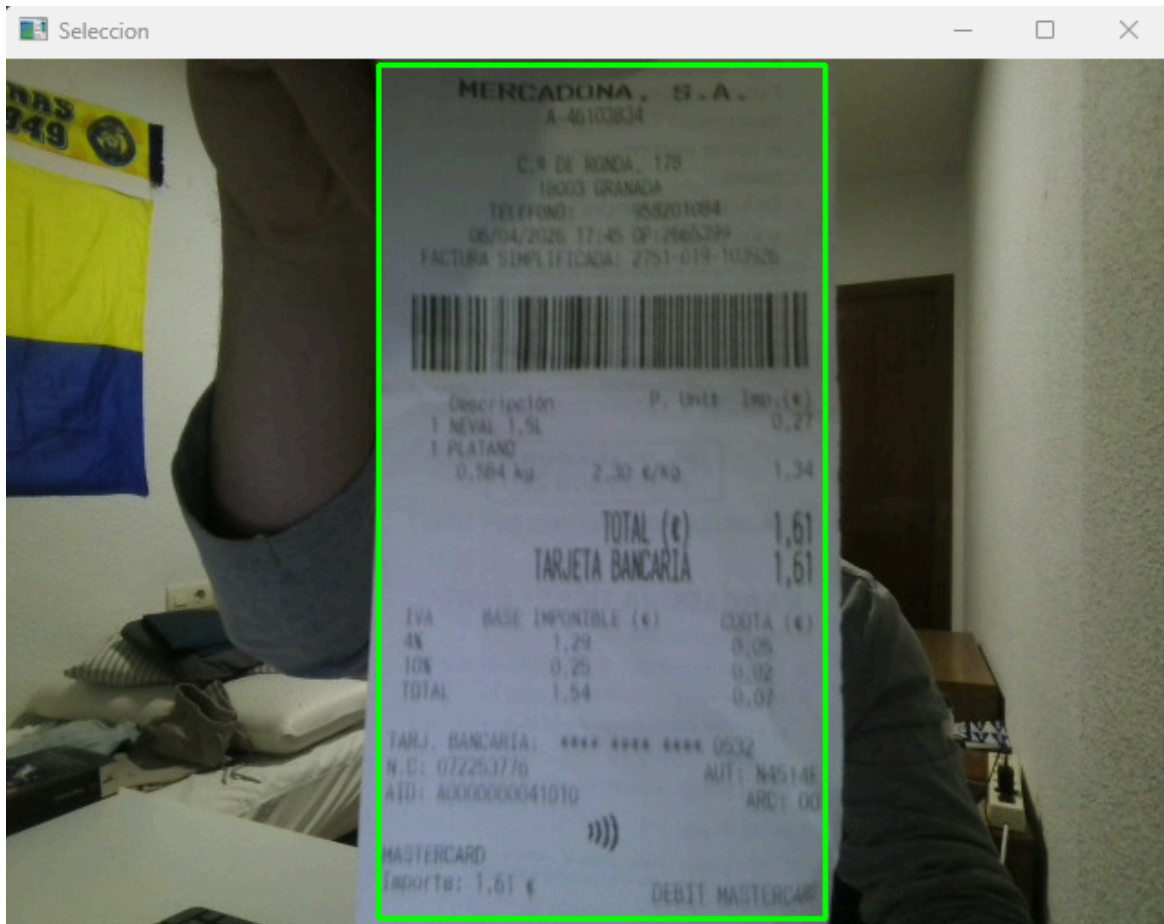
        # mejora ligera de contraste (NO agresiva)
        gray = cv2.convertScaleAbs(gray, alpha=1.3, beta=10)

        text = pytesseract.image_to_string(gray, config='--psm 6')

        print("Texto reconocido:")
        print(text)

    # Pulsar 'q' para salir
    if key == ord('q'):
        break
```

```
cap.release()
cv2.destroyAllWindows()
```



Texto reconocido:

Deser ipctén i :

; s) kg 2.30 eho 1.340

TO (¢) 1,6

| TARJETA MO 1,61

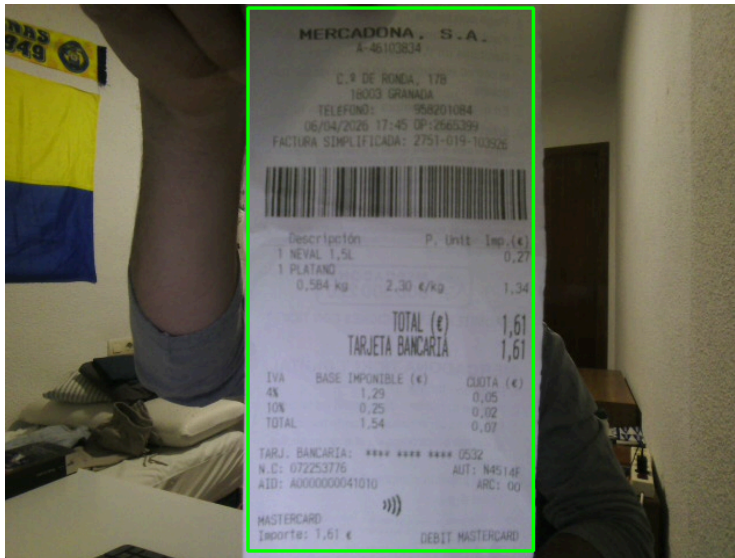
») Be

Al realizarlo de esta manera podemos observar que no identifica prácticamente los caracteres del recibo, a pesar de que antes si que lo hacía casi perfectamente, puedo que esto se deba al preprocesado que le he añadido a pesar de lo que comentábamos anteriormente, sin embargo, lo más probable es que esto sea debido a que la webcam no captura una imagen de buena calidad a la vez que el hecho de que haya muchas sombras, dificultando captar la imagen apropiadamente.

He probado a quitarle el poco preprocesado que le añadí en este caso a la imagen, donde la convertía a escala de grises y la agrandaba, y ahora le paso directamente el recorte a la Api:

```
cv2.imshow("Recorte", crop)

text = pytesseract.image_to_string(crop, config='--psm 6')
```



Texto reconocido:

Bes pe t6n =

ie ne is

TL (t) 1

IRENA wea 161

4

Py)

Aún así sigue sin detectarse prácticamente ningún caracter a pesar de que parece que la imagen se vea mejor sin el preprocesado.

Prueba Extra